

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – FUNCTION CALLING & STRUCTURED OUTPUT TASK

Course Code:	CSA65	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO2 – Precision (BL2, BL3)	Assessment:	Assessment Tool 3 – Function Calling & Structured Output Task
Weightage:	40% of CIA	Total Marks:	1 * 100 = 100 (1 Question1)
CO2 Statement:	<i>Apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems (BL2, BL3)</i>		
Student Name:	N Goutham	Reg. No.:	192472024
Section / Batch:	CSE-AI	Date:	04-08-2026

Instructions to Students

- Answer 1 question. Total: 100 Marks.
- Analyze the given scenario and identify the appropriate function(s), tool(s), parameters, and structured output required to solve the task.
- Design a valid function-calling schema with appropriate function names, parameters, data types, required fields, and descriptions based on the given requirements.
- Implement the function-calling workflow demonstrating the complete process: User Query → LLM → Function/Tool Call → Function Execution → Structured Response.
- Ensure that the generated output strictly follows the defined structured format or JSON schema and contains valid, relevant, and consistent information.
- Handle appropriate edge cases such as missing parameters, invalid inputs, ambiguous requests, incorrect data types, or unavailable information, and provide suitable responses without producing invalid function calls.
- Demonstrate the working implementation using suitable test cases, including normal inputs and at least one edge-case scenario.
- Clearly document the designed schema by explaining the purpose of each function, its parameters, data types, required fields, expected input, and expected output.
- Briefly justify the design decisions and explain how function calling and structured output improve the reliability, consistency, and usability of the LLM-based application.

Submission Requirements

Students should submit:

- **Source code / Jupyter Notebook**
- **Function-calling schema**
- **Structured output/JSON schema**
- **Execution screenshots**
- **Test cases and results**
- **Brief documentation explaining the schema and function-calling workflow**

Code of Conduct

I N Goutham/192472024(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N Goutham

5. Banking Assistant

A bank wants an AI assistant that can retrieve account balance and recent transactions.

Task: Design `get_account_balance()` and `get_transactions()` functions. Ensure that the output follows a predefined JSON schema and demonstrate appropriate handling of invalid account information.

Title

Banking Assistant using Function Calling and Structured Output

Aim

To design a Banking AI Assistant that retrieves account balance and recent transactions using function calling and returns the results in a structured JSON format while handling invalid account information.

Problem Statement

A bank wants an AI assistant that can:

- Retrieve customer account balance.
- Display recent transactions.
- Return responses in a predefined JSON format.
- Handle invalid or missing account information securely.

Function 1: `get_account_balance()`

Purpose

Retrieves the available balance of a customer's bank account.

Function Schema

```
def get_account_balance(  
    account_number: str  
)
```

Parameters

Parameter	Data Type	Required	Description
account_number	string	Yes	Customer Bank Account Number

JSON Output Schema :

```
{  
  "status": "success",  
  "account_number": "1234567890",  
  "account_holder": "Rahul Kumar",  
  "available_balance": 75890.50,  
  "currency": "INR"  
}
```

Function 2: get_transactions()

Purpose

Returns the recent transactions of a customer.

Function Schema

```
def get_transactions(  
    account_number: str,  
    limit: int  
)
```

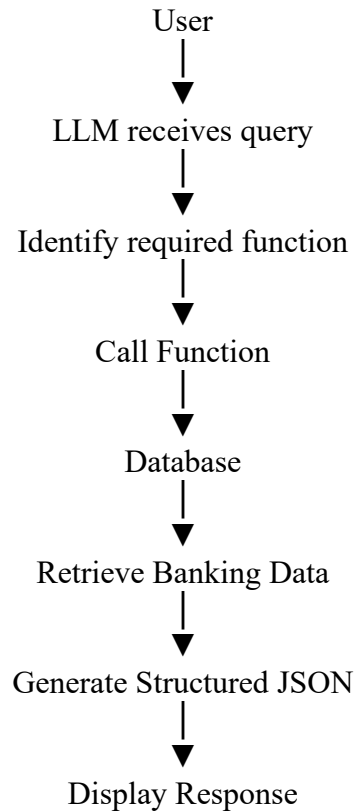
Parameters

Parameter	Type	Required	Description
account_number	string	Yes	Bank Account Number
limit	integer	Yes	Number of recent transactions

JSON Output Schema :

```
{  
  "status": "success",  
  "account_number": "1234567890",  
  "transactions": [  
    {  
      "date": "2026-08-01",  
      "type": "Debit",  
      "amount": 2500,  
      "description": "ATM Withdrawal"  
    },  
    {  
      "date": "2026-07-30",  
      "type": "Credit",  
      "amount": 15000,  
      "description": "Salary Credit"  
    }  
  ]  
}
```

Workflow :



Function Calling Demo

Scenario 1 :

User Query

Show my account balance.

LLM Decision

Calls

```
get_account_balance(  
    account_number="1234567890"  
)
```

Function Execution

Database returns

Balance = ₹75,890.50

Structured Response

```
{  
  "status": "success",  
  "account_number": "1234567890",  
  "available_balance": 75890.50,  
  "currency": "INR"
```

```
}
```

Scenario 2

User Query

Show my last 3 transactions.

LLM Decision

Calls

```
get_transactions(  
  account_number="1234567890",  
  limit=3  
)
```

Function Execution

Database returns

1. Salary Credit
2. ATM Withdrawal
3. UPI Payment

Structured Response :

```
{  
  "status":"success",  
  "transactions":[  
    {  
      "date":"2026-08-01",  
      "type":"Credit",  
      "amount":15000,  
      "description":"Salary Credit"  
    },  
    {  
      "date":"2026-07-31",  
      "type":"Debit",  
      "amount":2000,  
      "description":"ATM Withdrawal"  
    },  
    {  
      "date":"2026-07-30",  
      "type":"Debit",  
      "amount":500,  
      "description":"UPI Payment"  
    }  
  ]  
}
```

Edge Case Handling

Invalid Account Number

User Query

Show my balance.

Account Number

9999999999

Response

```
{  
  "status": "error",  
  "error_code": "INVALID_ACCOUNT",  
  "message": "The account number does not exist."  
}
```

Missing Account Number

User Query

Check my balance.

LLM Response

Please provide your account number to continue.

Invalid Transaction Limit

User Query

Show last -5 transactions.

Response

```
{  
  "status": "error",  
  "message": "Transaction limit must be greater than zero."  
}
```

Why Function Calling?

Function calling:

- Improves reliability by invoking predefined banking functions instead of generating unsupported values.
- Produces consistent JSON outputs suitable for banking applications.
- Separates natural language understanding from backend operations.
- Enables secure access to verified banking data.

Advantages

- Structured JSON responses.
- Reliable and predictable outputs.
- Easy integration with banking databases.
- Secure retrieval of customer information.
- Graceful handling of invalid or missing inputs.

Conclusion

The Banking Assistant uses function calling to retrieve account balances and recent transactions while returning structured JSON responses. The design supports clear function schemas, predictable outputs, and appropriate handling of invalid account information, meeting the assessment requirements.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Schema Design	30%	Correctly designs a function-calling schema and structured (JSON) output format	Mostly correct schema with minor issues	Schema has significant errors	Schema is fundamentally incorrect or missing
Output Validity	25%	LLM reliably produces valid structured output matching the schema	Mostly valid output with occasional errors	Output is frequently invalid or inconsistent	Output rarely or never matches the schema
Edge-Case Handling	20%	Handles edge cases (missing fields, ambiguous input) gracefully	Handles common cases well	Handles only the simplest cases	Fails on basic cases
Function-Call Flow Demo	15%	Clear demo of the function-call to structured-response flow	Demo covers the main flow	Demo is incomplete	No functional demo presented
Schema Documentation	10%	Well-documented schema design rationale	Adequate documentation	Minimal documentation	No documentation provided

Marks Evaluation:

Question No.	Schema Design(30%)	Output Validity (25%)	Edge-Case Handling (20%)	Function-Call Flow Demo(15%)	Schema Documentation (10%)	Total Marks (100)
Q1						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____